

2주차 과제

파이썬 머신러닝 완벽 가이드 2장, 3장 (2.6장, 3.6장 제외) 개념정리 및 필사

2장 사이킷런으로 시작하는 머신러닝

2.1 사이킷런 소개와 특징

사이킷런(scikit-learn): 파이썬 머신러닝 라이브러리 중 가장 많이 사용되는 라이브러리



사이킷런 특징

- 파이썬 기반의 다른 머신러닝 패키지도 사이킷런 스타일의 API 지향, 가장 쉽고 가장 파이썬스러운 API 제공
- 머신러닝을 위한 매우 다양한 알고리즘과 개발을 위한 편리한 프레임워크와 API 제공
- 오랜 기간 실전 환경에서 검증되었으며, 매우 많은 환경에서 사용되는 성숙한 라이브러리

2.2 첫번째 머신러닝 만들어 보기 - 불꽃 품종 예측하기

분류: 대표적인 지도학습 방법의 하나

cf. 지도학습: 학습을 위한 다양한 피처와 분류 결정값인 레이블 데이터로 모델을 학습한 뒤, 별도의 테스트 데이터 세트에서 미지의 레이블 예측

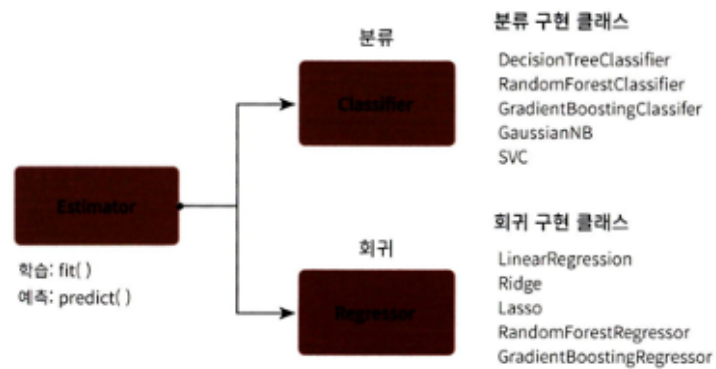
<코드 필사>

2.3 사이킷런의 기반 프레임워크 익히기

Estimator 이해 및 fit(), predict() 메서드

사이킷런은 API 일관성과 개발 편의성을 제공하기 위한 노력이 보이는 패키지
ML 모델 학습을 위해서: `fit()`, 학습된 모델의 예측을 위해 `predict()` 메서드 제공

▼ 분류



사이킷런의 주요 모듈

분류	모듈명	설명
예제 데이터	<code>sklearn.datasets</code>	사이킷런에 내장되어 예제로 제공하는 데이터 세트
피처 처리	<code>sklearn.preprocessing</code>	데이터 전처리에 필요한 다양한 가공 기능 제공(문자열을 숫자형 코드 값으로 인코딩, 정규화, 스케일링 등)
	<code>sklearn.feature_selection</code>	알고리즘에 큰 영향을 미치는 피처를 우선순위로 선택 작업 수행하는 다양한 기능 제공

피처 처리	<code>sklearn.feature_extraction</code>	텍스트 데이터나 이미지 데이터의 벡터화된 피처를 추출하는데 사용됨. 예를 들어 텍스트 데이터에서 Count Vectorizer나 Tfidf Vectorizer 등을 생성하는 기능 제공. 텍스트 데이터의 피처 추출은 <code>sklearn.feature_extraction.text</code> 모듈에, 이미지 데이터의 피처 추출은 <code>sklearn.feature_extraction.image</code> 모듈에 지원 API가 있음.
피처 처리 & 차원 축소	<code>sklearn.decomposition</code>	차원 축소와 관련한 알고리즘을 지원하는 모듈임. PCA, NMF, Truncated SVD 등을 통해 차원 축소 기능을 수행할 수 있음
데이터 분리, 검증 & 파라미터 튜닝	<code>sklearn.model_selection</code>	교차 검증을 위한 학습용/테스트용 분리, 그리드 서치(Grid Search)로 최적 파라미터 추출 등의 API 제공
평가	<code>sklearn.metrics</code>	분류, 회귀, 클러스터링, 페어와이즈(Pairwise)에 대한 다양한 성능 측정 방법 제공 Accuracy, Precision, Recall, ROC-AUC, RMSE 등 제공
ML 알고리즘	<code>sklearn.ensemble</code>	앙상블 알고리즘 제공 랜덤 포레스트, 에이다 부스팅, 그래디언트 부스팅 등을 제공
	<code>sklearn.linear_model</code>	주로 선형 회귀, 릿지(Ridge), 라쏘(Lasso) 및 로지스틱 회귀 등 회귀 관련 알고리즘을 지원. 또한 SGD(Stochastic Gradient Descent) 관련 알고리즘도 제공
	<code>sklearn.naive_bayes</code>	나이브 베이즈 알고리즘 제공. 가우시안 NB, 다항 분포 NB 등.
	<code>sklearn.neighbors</code>	최근접 이웃 알고리즘 제공. K-NN 등
	<code>sklearn.svm</code>	서포트 벡터 머신 알고리즘 제공
	<code>sklearn.tree</code>	의사 결정 트리 알고리즘 제공
유틸리티	<code>sklearn.cluster</code>	비지도 클러스터링 알고리즘 제공 (K-평균, 계층형, DBSCAN 등)
	<code>sklearn.pipeline</code>	피처 처리 등의 변환과 ML 알고리즘 학습, 예측 등을 함께 묶어서 실행할 수 있는 유틸리티 제공

내장된 예제 데이터 세트

▼ 분류나 회귀 연습용 예제 데이터

API 명	설명
<code>datasets.load_boston()</code>	회귀 용도이며, 미국 보스턴의 집 피쳐들과 가격에 대한 데이터 세트
<code>datasets.load_breast_cancer()</code>	분류 용도이며, 위스콘신 유방암 피쳐들과 악성/양성 레이블 데이터 세트
<code>datasets.load_diabetes()</code>	회귀 용도이며, 당뇨 데이터 세트
<code>datasets.load_digits()</code>	분류 용도이며, 0에서 9까지 숫자의 이미지 픽셀 데이터 세트
<code>datasets.load_iris()</code>	분류 용도이며, 붓꽃에 대한 피쳐를 가진 데이터 세트

- `fetch_covtype()`: 회귀 분석용 토지 조사 자료
- `fetch_20newsgroups()`: 뉴스 그룹 텍스트 자료
- `fetch_olivetti_faces()`: 얼굴 이미지 자료

- `fetch_lfw_prople()`: 얼굴 이미지 자료
- `fetch_lfw_pairs()`: 얼굴 이미지 자료
- `fetch_rcv1()`: 로이터 뉴스 말뭉치
- `fetch_mldata()`: ML 웹사이트에서 다운로드

▼ 분류와 클러스터링을 위한 표본 데이터 생성기

API 명	설명
<code>datasets.make_classifications()</code>	분류를 위한 데이터 세트를 만듭니다. 특히 높은 상관도, 불필요한 속성 등의 노이즈 효과를 위한 데이터를 무작위로 생성해 줍니다.
<code>datasets.make_blobs()</code>	클러스터링을 위한 데이터 세트를 무작위로 생성해 줍니다. 군집 지정 개수에 따라 여러 가지 클러스터링을 위한 데이터 세트를 쉽게 만들어 줍니다.

키는 보통 `data`, `target`, `target_name`, `feature_names`, `DESCR` 로 구성

- `data`: 피처의 데이터 세트
- `target`: 분류 시 레이블 값, 회귀일 때는 숫자 결괏값 데이터 세트
- `target_names`: 개별 레이블 이름
- `feature_names`: 피처의 이름
- `DESCR`: 데이터 세트에 대한 설명과 각 피처의 설명

2.4 Model Selection 모듈 소개

사이킷런의 `model_selection` 모듈: 학습 데이터와 테스트 데이터 세트를 분리하거나 교차 검증 분석 및 평가, 그리고 Estimator의 하이퍼 파라미터를 튜닝하기 위한 다양한 함수와 클래스를 제공.

학습/테스트 데이터 세트 분리 - `train_test_split()`

테스트 데이터 세트를 이용하지 않고 학습 데이터 세트로만 학습하고 예측하면, 정확도 100% ⇒ 분리가 필요한 이유

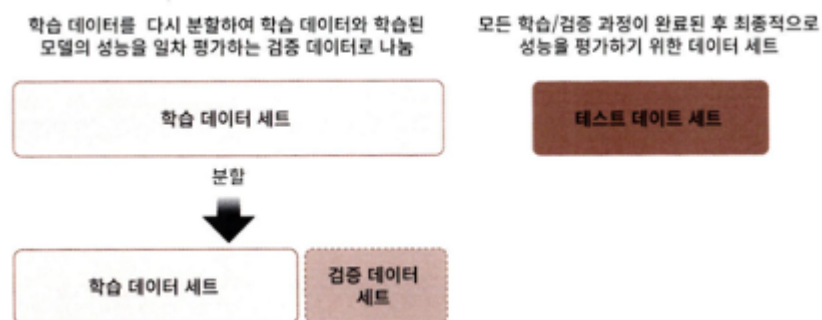
- `test_size`: 전체 데이터에서 테스트 데이터 세트 크기를 얼마로 샘플링할 것인가를 결정. 디폴트는 0.25
- `train_size`: 전체 데이터에서 학습용 데이터 세트 크기를 얼마로 샘플링할 것인가를 결정. `test_size` parameter 를 통상적으로 사용하기 때문에 `train_size`는 잘 활용
x

- **shuffle**: 데이터를 분리하기 전에 데이터를 미리 섞을지를 결정. 디폴트는 True. 데이터를 분산시켜서 좀 더 효율적인 학습 및 테스트 데이터 세트를 만드는 데 사용
- **random_state**: 호출할 때마다 동일한 학습/테스트용 데이터 세트를 생성하기 위해 주어지는 난수 값. `train_test_split()`는 호출 시 무작위로 데이터를 분리
→ `random_state`를 지정하지 않으면 수행할 때마다 다른 학습/테스트용 데이터를 생성.
- `train_test_split()`의 반환값은 튜플 형태. 순차적으로 학습용 데이터의 피쳐 데이터 세트, 테스트용 데이터의 피쳐 데이터 세트, 학습용 데이터의 레이블 데이터 세트, 테스트용 데이터의 레이블 데이터 세트가 반환

교차 검증

: 본고사를 치르기 전에 모의고사를 여러 번 보는 것

▼ ML 모델의 성능 평가 과정



• K 폴드 교차 검증

: 가장 보편적으로 사용되는 교차 검증 기법

▼ 5폴드 교차 검증



• Stratified K 폴드

: 불균형한 분포도를 가진 레이블(결정 클래스) 데이터 집합을 위한 K 폴드 방식

ex. 대출 사기 데이터 예측

(코드 필사에서 이어서 학습 진행!)

• 교차 검증을 보다 간편하게 - `cross_val_score()`

KFold로 데이터를 학습하고 예측하는 코드를 보면 먼저

1. 폴드 세트를 설정하고
2. for 루프에서 반복으로 학습 및 테스트 데이터의 인덱스를 추출한 뒤
3. 반복적으로 학습과 예측을 수행하고 예측 성능을 반환

• GridSearchCV - 교차 검증과 최적 하이퍼 파라미터 튜닝을 한 번에

: API 를 이용해 Classifier나 Regressor와 같은 알고리즘에 사용되는 하이퍼 파라미터를 순차적으로 입력하면서 편리하게 최적의 파라미터를 도출할 수 있는 방안 제공

<GridSearchCV 클래스의 생성자로 들어가는 주요 파라미터>

- estimator: classifier, regressor, pipeline
- param_grid: key+리스트 값을 가지는 딕셔너리가 주어짐. estimator의 튜닝을 위해 파라미터명과 사용될 여러 파라미터 값 지정
- scoring: 예측 성능을 측정할 평가 방법 지정. 보통은 사이킷런의 성능 평가 지표를 지정하는 문자열로 지정하나 별도의 성능 평가 지표 함수도 지정 가능
- cv: 교차검증을 위해 분할되는 학습/테스트 세트의 개수를 지정

- refit: 디폴트가 True로 생성 시 가장 최적의 하이퍼 파라미터를 찾은 뒤 입력된 estimator 객체를 해당 하이퍼파라미터로 재학습 시킴

<주요 칼럼별 의미>

- params: 수행할 때마다 적용된 개별 하이퍼 파라미터값을 나타냄
- rank_test_score: 하이퍼 파라미터별로 성능이 좋은 score 순위를 나타냄. 1이 가장 뛰어난 순위 ← 이때 파라미터가 최적의 하이퍼 파라미터
- mean_test_score: 개별 하이퍼 파라미터별로 CV의 폴딩 테스트 세트에 대해 총 수행한 평가 평균값

(코드 필사에서 이어서 학습 진행!)

2.5 데이터 전처리: 중요



결손값 (NAN, NULL)값은 허용 X

데이터 인코딩

• 레이블 인코딩

(코드 필사)

: 레이블 인코딩은 간단하게 문자열 값을 숫자형 카테고리 값으로 변환

but, 레이블 인코딩이 일괄 숫자로 변환이 되면서 몇몇 ML 알고리즘에는 이를 적용할 경우 예측 성능이 감소(숫자 값의 경우 크고 작음에 대한 특성이 작용)

• 원-핫 인코딩

: 피쳐 값의 유형에 따라 새로운 피쳐를 추가해 고유 값에 해당하는 칼럼에만 1을 표시하고 나머지 칼럼에는 0을 표시

▼ 원본 데이터를 원-핫 인코딩으로 변환하는 모습

원본 데이터		원-핫 인코딩					
상품 분류		상품분류_ TV	상품분류_ 냉장고	상품분류_ 믹서	상품분류_ 선풍기	상품분류_ 전자렌지	상품분류_ 컴퓨터
TV		1	0	0	0	0	0
냉장고	→	0	1	0	0	0	0
전자렌지		0	0	0	0	1	0
컴퓨터	→	0	0	0	0	0	1
선풍기		0	0	0	1	0	0
선풍기		0	0	0	1	0	0
믹서		0	0	1	0	0	0
믹서		0	0	1	0	0	0

(코드 필사)

• 피쳐 스케일링과 정규화

1. 표준화: 데이터의 피쳐 각각이 평균이 0이고, 분산이 1인 가우시안 정규 분포를 가진 값으로 변환하는 것을 의미

$$x_{i_new} = \frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$$

2. 정규화: 서로 다른 피쳐의 크기를 통일하기 위해 크기를 변환해주는 개념

$$x_{i_new} = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

cf. 사이킷런의 전처리에서 제공하는 Normalizer 모듈과 일반적인 정규화는 약간의 차이가 존재: 개념은 동일하나, 사이킷런의 Normalizer 모듈은 선형대수에서의 정규화 개념이 적용, 개별 벡터의 크기를 맞추기 위해 변환하는 것을 의미.

→ 개별 벡터를 모든 피쳐 벡터의 크기로 나눠 줌

$$x_{i_new} = \frac{x_i}{\sqrt{x_i^2 + y_i^2 + z_i^2}}$$

• StandardScaler

: 표준화를 쉽게 지원하기 위한 클래스 → 개별 피처를 평균이 0, 분산이 1인 값으로 변환.

- **MinMaxScaler**

: 데이터값을 0과 1사이의 범위 값으로 변환 (음수 값 존재시, -1에서 1값으로 변환), 데이터의 분포가 가우시안 분포가 아닐 경우 Min, Max Scale을 적용 가능

학습 데이터와 테스트 데이터의 스케일링 변환 시 유의점

StandardScaler 또는 MinMaxScaler와 같은 Scaler 객체를 이용해 데이터의 스케일링 변환 시 fit(), transform(), fit_transform() 메소드 이용

→ 일반적으로 fit()은 데이터 변환을 위한 기준 정보 설정을 적용, transform()은 이렇게 설정된 정보를 이용해 데이터를 변환, fit_transform()은 fit()과 transform()을 한번에 적용하는 기능 수행

학습 데이터 세트와 테스트 데이터 세트에 이 fit()과 transform()을 적용할 때 주의가 필요.

Scaler 객체를 이용해 학습 데이터 세트로 fit()과 transform()을 적용 → 테스트 데이터 세트로 다시 fit()을 수행 x, 학습 데이터 세트로 fit()을 수행한 결과를 이용해 transform() 변환을 적용해야 한다는 것

⇒ 학습 데이터로 fit()이 적용된 스케일링 기준 정보를 그대로 테스트 데이터에 적용해야 하며, 그렇지 않고 테스트 데이터로 다시 새로운 스케일링 기준 정보를 만들게 되면 학습 데이터와 테스트 데이터의 스케일링 기준 정보가 서로 달라지기 때문에 올바른 예측 결과를 도출하지 못할 수도 있음.

2.7 정리

1. 사이킷런

매우 많은 머신러닝 알고리즘 제공+쉽고 직관적인 API 프레임워크+편리고 다양한 모듈 지원등으로 파이썬 계열의 대표적인 머신러닝 패키지로 자리 잡음

⇒ 많은 머신러닝 클래스와 다양한 지원 모듈과 더불어 사이킷런의 직관적인 API는 머신러닝 애플리케이션을 쉽게 구현해줌

2. 머신러닝 애플리케이션

데이터의 가공 및 변환 과정의 전처리 작업, 데이터를 학습 데이터와 테스트 데이터로 분리하는 데이터 세트 분리 작업을 거친 후에 학습 데이터를 기반으로 머신러닝 알고리즘을 적용해 모델 학습, 학습된 모델을 기반으로 테스트 데이터에 대한 예측 수행, 예측된 결과값을 실제 결과값과 비교해 머신러닝 모델에 대한 평가를 수행하는 방식으로 구성됨.

3. 데이터의 전처리 작업

오류 데이터의 보정 or 결손값(NULL) 처리 등의 다양한 데이터 클렌징 작업, 레이블 인코딩이나 원-핫 인코딩과 같은 인코딩 작업, 그리고 데이터의 스케일링/정규화 작업 등으로 머신러닝 알고리즘이 최적으로 수행될 수 있게 데이터를 사전 처리

머신러닝 모델은 학습 데이터 세트로 학습한 뒤 반드시 별도의 테스트 데이터 세트로 평가되어야 함.

테스트 데이터의 건수 부족이나 고정된 테스트 데이터 세트를 이용한 반복적인 모델의 학습과 평가는 해당 테스트 데이터 세트에만 치우친 빈약한 머신러닝 모델을 만들 가능성이 높음.

→ 해결방법: 여러 개의 폴드 세트로 분리해 교차 검증 수행 가능⇒다양한 클래스와 함수 제공

3장 평가



머신러닝의 프로세스

데이터 가공/변환 → 모델 학습/예측 → **평가**

머신러닝 모델은 여러가지 방법으로 예측 성능을 평가할 수 있다.

회귀

대부분 실제값과 예측값의 오차 평균값에 기반

ex. 오차에 절대값을 씌운 뒤 평균 오차를 구함, 제곱 값에 루트를 씌운 뒤 평균 오차를 구함

분류

일반적으로는 실제 결과 데이터와 예측 결과 데이터가 얼마나 정확하고 오류가 적게 발생하는가에 기반.

정확도만 가지고 판단시 오류 발생 가능!

분류의 성능 평가 지표

- 정확도
- 오차행렬
- 정밀도
- 재현율
- F1 스코어
- ROC AUC

⇒ 이진 분류에서 더욱 중요(멀티 분류에도 적용)

3.1 정확도(Accuracy)

= (예측 결과가 동일한 데이터 건수) / (전체 예측 데이터 건수)

(코드 필사)

3.2 오차 행렬

: 학습된 분류 모델이 예측을 수행하면서 얼마나 헛갈리고 있는지도 함께 보여주는 지표

▼ 오차 행렬

- TN: 예측값을 Negative 값 0으로 예측했고 실제 값 역시 Negative 값 0
- FP: 예측값을 Positive 값 1로 예측했는데 실제 값은 Negative 값 0
- FN: 예측값을 Negative 값 0으로 예측했는데 실제 값은 Positive 값 1
- TP: 예측값을 Positive 값 1로 예측했는데 실제 값 역시 Positive 값 1

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	TN (True Negative)	FP (False Positive)
	Positive(1)	FN (False Negative)	TP (True Positive)

3.3 정밀도와 재현율

- 정밀도 = $TP / (FP + TP)$

→ 상대적으로 더 중요한 지표인 경우: 실제 Negative 음성인 데이터 예측을 Positive 양성으로 잘못 판단하게 되면 업무상 큰 영향이 발생하는 경우

- 재현율 = $TP / (FN + TP)$

→ 상대적으로 더 중요한 지표인 경우: 실제 Positive 양성인 데이터 예측을 Negative로 잘못 판단하게 되면 업무상 큰 영향이 발생하는 경우

정밀도와 재현율의 맹점

정밀도가 100%가 되는 방법

: 확실한 기준이 되는 경우만 Positive로 예측, 나머지는 모두 Negative로 예측

재현율이 100%가 되는 방법

: 모든 환자를 Positive로 예측

⇒ 어느 한쪽만 참조하면 극단적인 수치 조작이 가능.

3.4 F1 스코어

: 정밀도와 재현율을 결합한 지표

▼ 공식

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

3.5 ROC 곡선과 AUC

: 이진 분류의 예측 성능 측정에서 중요하게 사용되는 지표

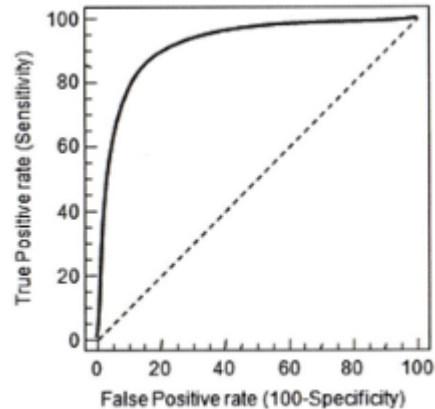
- ROC 곡선(Receiver Operation Characteristic Curve)

: FRR이 변할 때 TPR이 어떻게 변하는지를 나타내는 곡선

- 민감도(TPR)는 실제값 Positive(양성)가 정확히 예측돼야 하는 수준을 나타냄 (질병 o → 질병 o로 판정)
- 특이성(TNR)은 실제값 Negative(음성)가 정확히 예측돼야 하는 수준을 나타냄 (질병 x → 질병 x로 음성 판정)

$$FRR = FP / (FP+TN) = 1 - TNR = 1 - \text{특이성}$$

▼ ROC 곡선의 예



〈 ROC 곡선 예시 〉

3.7 정리

1. 왜 정확도만으로는 부족한가

- 배경: 정확도, 오차 행렬, 정밀도, 재현율, F1 스코어, ROC-AUC 학습
- 핵심 문제: 클래스 불균형 (레이블 값이 한쪽으로 치우친 경우)
- 결론: 불균형 데이터에서는 정확도만으로 모델 성능 평가 불가

2. 오차 행렬 (Confusion Matrix)

- 구성: 실제 클래스(Negative/Positive) × 예측 클래스(True/False)
- 4분면: TN, FP, FN, TP
- 역할: 이 4가지 값을 조합해 정확도, 정밀도, 재현율, F1 등을 산출 → 오류 양상 파악 가능

3. 정밀도(Precision) vs 재현율(Recall)

구분	내용
공통점	Positive 데이터 세트 예측 성능에 초점
재현율이 중요한 경우	암 양성 예측 모델처럼 실제 Positive를 Negative로 잘못 판단(FN)하면 업무상 타격이 큰 경우
실무 팁	분류 업무 특성에 따라 정밀도/재현율 중 강조할 지표를 정하고, Threshold(임계값) 조정으로 수치를 조절

4. F1 스코어

- 정의: 정밀도와 재현율을 결합한 지표

- 특징: 두 지표가 어느 한쪽으로 치우치지 않을 때 높은 값

5. ROC-AUC

- 정의: 이진 분류에서 가장 많이 사용되는 성능 평가 지표
- AUC: ROC 곡선 밑의 면적, 1에 가까울수록 좋은 수치